

Proof Structure and Neural Theorem Proving

Riyaz Ahuja

Table of Contents

On Recent NTP Results

Is “Structure” Useful?

Proof Optimization

Previous Work

ImProver²

Formal Proof Structure

Results

Bootstrapping SLMs with Off-Policy Expert Iteration

“Structure” is Useful.

In the beginning*, we had MCTS:

HTPS:

- First really influential AlphaZero-style search procedure for neural theorem proving
 - Tree search over proof states with value/policy network guidance, and online training from the search itself
- Improved SoTA on miniF2F to 41%

BFS-Prover:

- Instead of MCTS or HTPS, uses a best first search over proof states. Given as an inference time algorithm over DeepSeek-Prover-V1.5 and scored via DPO on proof traces.
- Beats its DSP1.5 baseline by 7-10 points on MiniF2F (70.83%)

In the beginning*, we had MCTS:

HTPS:

```
1 theorem imo_1964_p1_2 (n : N) :  $\neg 7 \mid 2^n + 1 :=$ 
2 begin
3   rw nat.dvd_iff_mod_eq_zero,
4   rewrite [nat.add_mod, nat.mod_eq_of_lt],
5   obviously,
6   apply nat.strong_induction_on n,
7   induction n,
8   {
9     intros n IH,
10    cases n,
11    norm_num,
12    cases n,
13    norm_num,
14    rw [nat.succ_eq_add_one, pow_succ],
15    rw [nat.succ_eq_add_one, pow_succ],
16    induction n,
17    norm_num,
18    rw [nat.succ_eq_add_one, pow_succ],
19    norm_num [nat.mul_mod, ←mul_assoc],
20    contrapose! IH,
21    refine ⟨n_n, nat.lt_succ_iff.mpr _, IH⟩,
22    exact nat.le_succ_of_le (nat.le_succ _),
23  },
24  exact n_ih,
25 end
```

BFS-Prover:

```
theorem imo_1962_p2 (x : R)
(h0 :  $0 \leq 3 - x$ )
(h1 :  $0 \leq x + 1$ )
(h2 :  $1/2 < \sqrt{3 - x} - \sqrt{x + 1}$ ) :
-1 ≤ x and x < 1 - sqrt(31)/8 := by {
  constructor
  linarith
  rw [← sub_pos]
  field_simp [Real.sqrt_lt] at h2 |-
  apply lt_of_le_of_lt
  rw [mul_comm]
  rw [sub_eq_add_neg]
  apply lt_of_le_of_lt
  rw [← lt_sub_iff_add_lt]
  ring_nf
  rw [← lt_sub_iff_add_lt]
  linarith [Real.sq_sqrt (by linarith :  $0 \leq 1 + x$ )]
  rw [Real.sqrt_lt (by norm_num)]
  rw [Real.sqrt_lt] <=> nlinarith
  norm_num at this
}
```

And simple LLM provers:

DeepSeek-Prover-v1.5:

- Pre-train on informal math + formal math.
- Fine-tune on formal theorems/proofs from DeepSeek-Prover-V1.
- RL from Lean compilation signal.
- Also has a MCTS variant for test time exploration.
- SoTA on MiniF2F at 63.5%

```
theorem amc12_2000_p12 (a m c : N) (h₀ : a + m + c = 12) :  
  a * m * c + a * m + m * c + a * c ≤ 112 := by  
  /-  
  Given nonnegative integers A, M, and C such that A+M+C=12, we need to find  
  the maximum value of A·M·C+A·M+M·C+A·C.  
  To solve this, we can use the fact that the expression A·M·C+A·M+M·C+A·C  
  is a polynomial in terms of A, M, and C. We can try different values of A, M,  
  and C that sum to 12 and compute the value of the polynomial for each  
  combination. The maximum value of the polynomial will then be the solution.  
  -/  
  -- We use a lemma that states if the sum of three numbers is less than or  
  -- equal to a certain value, then the product of these numbers is also less  
  -- than or equal to a certain value.  
  have h₁ : a ≤ 12 ∧ m ≤ 12 ∧ c ≤ 12 := by  
    constructor  
    · -- Since a + m + c = 12, a cannot be greater than 12.  
      omega  
    · constructor  
      · -- Similarly, m cannot be greater than 12.  
        omega  
      · -- And c cannot be greater than 12.  
        omega  
  -- Using the lemma, we compute the value of the polynomial for different  
  -- values of a, m, and c that sum to 12.  
  cases' h₁ with h₁_left h₁_right  
  cases' h₁_right with h₁_middle h₁_right  
  -- We use interval_cases to try different values of a, m, and c.  
  interval_cases a <|> interval_cases m <|> interval_cases c <|>  
  -- For each combination, we compute the value of the polynomial and check  
  -- if it is less than or equal to 112.  
  simp_all (config := {decide := true})
```

And then, less simple ones as well:

AlphaProof:

- Autoformalize ~100M formal theorems, and use them in an EI+RL loop: generate proofs with search, train a policy/value network on successful (compiling) ones.
- Test-time online RL to adapt to a given problem at test time.
- 4/6 IMO
- (Not as much informal reasoning; is a 3B encoder-decoder model!)

```

import Mathlib
open scoped Nat

/--
Determine all pairs (a,b) of positive integers for which there exist positive integers g and N such that

 $\gcd(a^n + b, b^n + a) = g$ 

holds for all integers n ≥ N. (Note that gcd(x,y) denotes the greatest common divisor of integers x and y.)
-/
theorem IMO_2024_p2 : { (a,b) | 0 < a ∧ 0 < b ∧ ∃ g,N, 0 < g ∧ 0 < N ∧ ∀ n ≥ N, Nat.gcd (a ^ n + b) (b ^ n + a) = g } =
{ (1,1) } := by
induction! (0) => 2
- use Set.eq_singleton_iff_unique_mem.2 (?.>, λb g=>bby.contra8 $ g.2.2.rec λY S i =>S.rec λX D =>?).
- exact (by left|by left|2,3,by simp_all)
have h.b.1*b.2|Y?=?
- suffices: b.1=b.2
norm_num [b.ext_iff,<-D.2.2 L,this] at *
use pow.lt.pow (g.1.nat_succ.le, ltf.ne' i) (by left)|.ne' (D.2.2 - L.le_succ)
suffices:b.1+b.2|b.fst*(2*L)+b.2*A(b).fst<(b).snd | b.snd*(2*L)+b.1
- suffices:b.1*(2*(b.1+b.2))+b.2*(2*(b.1+b.snd))
- norm_num[Nat.add_mod,pow_mul,this,Nat.dvd_iff_mod_eq_zero,Nat.pow_mod] at *
norm_num[add_comm,b.ext_iff,sq _ ,←Nat.pow_mod,←Nat.dvd_iff_mod_eq_zero] at *
zify at *
cases this.1.sub this.2.with|_ Z=>nlinarith [(by (nlinarith): Z=0)]
apply!Nat.modEq.of_dvd
use (b.snd)-b.fst , (by-iring: ((b.snd):Z)+2-b.fst*(2*(b.fst+(b.2))-*))
norm_num(2).le_mul_of_pos_left,Nat.gcd dvd, ←D.2.2 (2*),this.trans,(D.Right.1 :..)
suffices:b.1+b.2|b.1*(2*L)+b.2 A b.1+b.2 |b.snd*(2*L)+b.1
- exact D.2.2 (2*L) ((le_mul_of_one_le_left' (by decide))) >dvd_gcd (this.left) (this).2
ex falso
suffices:b.1+b.2|Y
- suffices:b.1≠(b.1+b.2+1)%((b.1+b.2+1))=1%(b.1+b.2+1) Ab,2^q (b.1+b.snd+1)%((b.1+1)*↑(b.snd)+1)=1% (b.1+b.snd+1)
- absurd D.2.2 (q (b.1+b.2+1))*L (by nlinarith [(b.fst ≡ b.2+1).totient_pos].2 ↑Fin.size_pos'])
apply mt (<-Nat.gcd_dvd -)
obv-obv-absurd (<-|V,trans H.1) (λx>absurd (<-|V,trans H.2) ?-)
norm_num[pow_mul,b.ext_iff,lt,Mod_eq_of_lt,g.symm,this.Nat.add_mod,Nat.dvd_iff_mod_eq_zero,Nat.pow_mod] at (i)v-
norm_num[add_comm,pow_mul,←Nat.dvd_iff_mod_eq_zero] at *
contra pose! i
zify at *
repeat use by nlinarith[Int.le.of_dvd (by nlinarith) v,Int.le.of_dvd (by nlinarith) i]
repeat use (Nat.ModEqEq.pow.totient (by norm_num))
by contr! H
suffices:b.1≠q ((b.1+b.2+1)%((b.1+b.2+1))=1%(b.1+b.2+1) Ab,2^q (b.1+b.2+1)%((b.1+b.2+1))=1%(b.fst + ↑(b.snd)+1)
- simp_all
suffices:b.1+b.2+1|b.1(q ((b.1+b.2+1)*(L+1)-1)+b.2 Ab.1+b.2+1|b.2(q (b.1+b.2+1)*(L+1)-1)+(b.fst)
- use HS.D.2.2 (q - *(L+1)-1) (L.le.sub.of_add.le (by nlinarith)[(b.1+b.2+1).totient_pos].2
- (Nat.succ_pos'.1))(<-(Nat.dvd_gcd) (this).1) at this
cases B:Nat.exists_eq_add_of_lf5 ((b.1+b.2+1).totient_pos).2 (by continuity)
norm_num[m*,g,q,-=,]
- mul_Add,Nat.pow_mod,(1).mod_eq_of_lt,pow_Add,Nat.add_mod,pow_mul,Nat.dvd_iff_mod_eq_zero,Nat.mul_mod] at this-
simp_all
suffices:b.1+b.2+1|b.1*( (b.1%(b.1+1) + (b.snd+1) : _) )^NAt.(b.snd) Λ(b.fst + ↑(b.snd) + 1)|(b).snd*(
- (b.snd*(b.fst + ↑(b.snd) + 1))^NAt.(b.fst)
- norm_num[←Nat.dvd_iff_mod_eq_zero,g,(1).mod_eq_of_lt,Nat.dvd_mul] at this-
exists?&?
- cases this.1.with|_ Q=>simp_all[(Q.dvd_gcd r.1 (·,·),symm r.right.choose_spec.2)),antisymm]
cases this.2.with|_ F=>simp_all[(F.dvd_gcd X.1 (·,·),symm X.2.right.choose_spec.2)].antisymm]
simp_all[mul_comm,mul_add,add_comm,Nat.add_mod,Nat.dvd_iff_mod_eq_zero]
repeat use (Nat.ModEqEq.pow.totient (by . . norm_num))

```

Then, came along subgoal decomposition:

DeepSeek-Prover-v2:

- Cold-start via subgoal decomposition: prompt DeepSeek-V3 to decompose problems and sketch their proofs. Use this to build a (subgoal, proof, CoT) dataset.
- Initialize DSP2 from this and then RL on Lean compiler feedback.
- 88.9% MiniF2F (SoTA) and 49 Putnam problems solved.

Kimina-Prover:

- Scaled up Qwen2.5-72B for NTP, introducing a structured formal reasoning “pattern”
 - Model CoT decomposes proof into a stepwise reasoning trace in Lean
 - RL optimizes the pattern
- 80.7% MinF2F (SoTA)

Then, came along subgoal decomposition:

DeepSeek-Prover-v2:

```
theorem imo_1992_p1 (p q r : Z) (h0 : 1 < p ∧ p < q ∧ q < r)
  (h1 : (p - 1) * (q - 1) * (r - 1) | p * q * r - 1) :
  (p, q, r) = (2, 4, 8) ∨ (p, q, r) = (3, 5, 15) := by
  have h2 : p ≥ 2 := by
    linarith [h0.1]

  have h3 : q ≥ 3 := by
    by_contra h
    have h4 : q < 2 := by linarith
    have h5 : q ≥ 2 := by linarith [h0.2.1]
    have h6 : q = 2 := by linarith
    have h7 : p < 2 := by linarith [h0.2.1]
    have h8 : p ≥ 2 := by linarith [h0.1]
    linarith

  have h4 : r ≥ 4 := by
    by_contra h
    have h5 : r < 3 := by linarith
    have h6 : r ≥ 3 := by linarith [h0.2.2]
    have h7 : r = 3 := by linarith
    have h8 : q < 3 := by linarith [h0.2.2]
    have h9 : q ≥ 3 := by linarith [h3]
    linarith

  have h5 : (p, q, r) = (2, 4, 8) ∨ (p, q, r) = (3, 5, 15) := by
    have h6 : (p - 1 : Z) > 0 := by
      linarith
    have h7 : (q - 1 : Z) > 0 := by
      linarith
    have h8 : (r - 1 : Z) > 0 := by
      linarith
    have h9 : (p - 1 : Z) * (q - 1 : Z) * (r - 1 : Z) > 0 := by positivity
    have h10 : ∃ (k : Z), (p * q * r - 1 : Z) = k * ((p - 1 : Z) * (q - 1 : Z) * (r - 1 : Z)) := by
      obtain (k, hk) := h1
      refine (k, _)
      linarith
    obtain (k, hk) := h10
    have h11 : k > 0 := by
      by_contra h11
      have h12 : k ≤ 0 := by linarith
      have h13 : (p * q * r - 1 : Z) ≤ 0 := by
        linarith [mul_nonneg (sub_nonneg.mpr h2) (sub_nonneg.mpr h3),
          mul_nonneg (sub_nonneg.mpr h2) (sub_nonneg.mpr h4),
          mul_nonneg (sub_nonneg.mpr h3) (sub_nonneg.mpr h4)]
      have h14 : (p * q * r : Z) ≤ 1 := by linarith
      have h15 : (p : Z) ≥ 2 := by linarith
      have h16 : (q : Z) ≥ 3 := by linarith
      have h17 : (r : Z) ≥ 4 := by linarith
      have h18 : (p * q * r : Z) ≥ 2 * 3 * 4 := by
        have h19 : (p : Z) * q ≥ 2 * 3 := by linarith
        linarith
      have h20 : k ≤ 3 := by
        by_contra h20
        have h21 : k ≥ 4 := by linarith
        have h22 : (p - 1 : Z) ≥ 1 := by
          linarith
        have h23 : (q - 1 : Z) ≥ 2 := by
          linarith
        have h24 : (r - 1 : Z) ≥ 3 := by
          linarith
```

Kimina-Prover:

```
have h_tan : Real.tan x = Real.sin x / Real.cos x := by
  rw [Real.tan_eq_sin_div_cos]

have h3 : Real.sin x ^ 2 + Real.cos x ^ 2 = 1 := Real.sin_sq_add_cos_sq x

have h4 : 1 / Real.cos x + Real.tan x = 22 / 7 := h0
  rw [h_tan] at h4
have h5 : 1 / Real.cos x + Real.sin x / Real.cos x = 22 / 7 := h4
have h6 : (1 + Real.sin x) / Real.cos x = 22 / 7 := by
  have h7 : Real.cos x ≠ 0 := h2
  field_simp [h7] at h5
  nlinarith

have h7 : 7 * (1 + Real.sin x) = 22 * Real.cos x := by
  have h8 : Real.cos x ≠ 0 := h2
  field_simp at h6
  nlinarith

have h_sin : Real.sin x = (22 * Real.cos x - 7) / 7 := by linarith

have h8 : Real.sin x ^ 2 + Real.cos x ^ 2 = 1 := Real.sin_sq_add_cos_sq x
have h9 : Real.sin x = (22 * Real.cos x - 7) / 7 := h_sin

have h_cos_sq : Real.cos x ^ 2 = (308 / 533) ^ 2 := by
  have h10 : Real.cos x ≠ 0 := h2
  have h11 : Real.sin x = (22 * Real.cos x - 7) / 7 := h_sin
  rw [h11] at h3
  have h12 : ((22 * Real.cos x - 7) / 7) ^ 2 + Real.cos x ^ 2 = 1 := by linarith
  have h13 : (22 * Real.cos x - 7) ^ 2 + 7 * Real.cos x ^ 2 = 49 := by
    nlinarith
  have h14 : (Real.cos x) * (533 * Real.cos x - 308) = 0 := by
    ring_nf at h13
    linarith
  cases' (mul_eq_zero.mpr h14) with h_cos h15
  · -- Real.cos x = 0
    ex falso
  · exact h10 (by linarith)
  · -- 533 * Real.cos x - 308 = 0
    have h_cos : Real.cos x = 308 / 533 := by linarith
    nlinarith [h_cos]
have h_cos : Real.cos x = 308 / 533 := by nlinarith [h_cos_sq]

have h_sin' : Real.sin x = 435 / 533 := by
  have h9 : Real.sin x = (22 * Real.cos x - 7) / 7 := h_sin
  rw [h9, h_cos]
  norm_num

have h_csc_cot : (1 / Real.sin x : R) + (1 / Real.tan x : R) = (29 / 15 : R) := by
  have h10 : Real.sin x ≠ 0 := by
    by_contra h
    have h11 : Real.sin x = 0 := by linarith
    rw [h11] at h3
    have h12 : Real.cos x ^ 2 = 1 := by nlinarith
    have h13 : Real.cos x = 1 ∨ Real.cos x = -1 := by
      have h14 : Real.cos x ^ 2 - 1 = 0 := by linarith
      have h15 : (Real.cos x - 1) * (Real.cos x + 1) = 0 := by
        ring_nf at h14
        linarith
      cases (mul_eq_zero.mpr h15) with
      | inl h16 => left; linarith
      | inr h17 => right; linarith
    cases h13 with
    | inl h14 =>
      have h15 : Real.cos x = 308 / 533 := h_cos
      linarith
```


And self-play:

STP-Prover:

- Conjecturer-Prover loop, where model generates its own curriculum of new theorems:
 - Conjecturer proposes new conjectures just beyond the current prover's reach
 - Prover tries to prove these theorems with expert iteration.
- 8 putnam problems and 65% MiniF2F

```
have : p^2 % 24 = (p % 24)^2 % 24 := by
rw [Nat.pow_mod]
have : q^2 % 24 = (q % 24)^2 % 24 := by
rw [Nat.pow_mod]
rw [← Nat.mod_add_div p 24, ← Nat.mod_add_div q 24]
ring_nf
norm_num
have : p % 24 - 1 ∨ p % 24 - 5 ∨ p % 24 - 7 ∨ p % 24 - 11 ∨ p % 24 - 13 ∨ p % 24 - 17
  ∨ p % 24 = 19 ∨ p % 24 = 23 := by
have := hp.eq_one_or_self_of_dvd 2
have := hp.eq_one_or_self_of_dvd 3
have := hp.eq_one_or_self_of_dvd 4
have := hp.eq_one_or_self_of_dvd 5
omega
have : q % 24 = 1 ∨ q % 24 = 5 ∨ q % 24 = 7 ∨ q % 24 = 11 ∨ q % 24 = 13 ∨ q % 24 = 17
  ∨ q % 24 = 19 ∨ q % 24 = 23 := by
have := hq.eq_one_or_self_of_dvd 2
have := hq.eq_one_or_self_of_dvd 3
have := hq.eq_one_or_self_of_dvd 4
have := hq.eq_one_or_self_of_dvd 5
omega
rcases this with (h | h | h | h | h | h | h | h) <|> rcases this with (h' | h' | h' | h' |
  h' | h' | h' | h') <|>
simp [h, h', pow_two, Nat.mul_mod, Nat.add_mod, Nat.mod_self, zero_add,
  Nat.zero_mul, Nat.mod_one, Nat.one_mul, Nat.mod_add_mod]
<|> omega
```

Goedel-Prover-v2:

- Goedel-Prover-v1 used a large autoformalization stage and expert iteration - data scaling.
- V2 improves this by generating synthetic tasks of increasing difficulty to train harder prover, and then use Lean feedback to self-correct within a trajectory.
- Run this pipeline in a standard RL + Expert iteration pipeline.
- 84.6% MiniF2F and 86 Putnam Problems.

And now at SoTA:

Seed-Prover:

- Builds on previous work heavily with large emphasis on lemma and conjecture generation and iterative refinement.
- Saturates MiniF2F, Gold IMO (5/6), >50% on PutnamBench.

Hilbert:

- Agentic framework - General LLM for informal reasoning, and a formal prover LLM.
- Recursively decomposes the problem into subgoals, and routes the subgoals to either the prover or informal engine.
- Saturates MiniF2F, solves 70% of PutnamBench.

```
theorem h_divides_ancil2b_2002_p4 (n : ℕ) (h₀ : 0 < n) (h₁ : (1 / 2 + 1 / 3 + 1 / 7 + 1 / ↑n : ℚ).den = 1) (h_sum_is_frac :  
  → (1 / 2 + 1 / 3 + 1 / 7 + 1 / ↑n : ℚ) = (↑(41 * n + 42)) / ↑(42 * n)) : 42 * n ∣ 41 * n + 42 := by  
  have h₂ : ((↑(41 * n + 42) : ℚ) / ↑(42 * n)).den = 1 := by  
    have h₃ : ((↑(41 * n + 42) : ℚ) / ↑(42 * n)) = (1 / 2 + 1 / 3 + 1 / 7 + 1 / ↑n : ℚ) := by  
      rw [h_sum_is_frac]  
    <|> field_simp <|> ring_nf <|> norm_cast  
    <|> field_simp <|> ring_nf <|> norm_cast  
    rw [h₃]  
  exact h₁  
  
  have h₄ : (42 * n : ℤ) ∣ (41 * n + 42 : ℤ) := by  
    have h₅ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).den = 1 := by  
      norm_cast at h₅  
    <|> simp [div_eq_mul_inv] using h₅  
    have h₆ : ∃ (k : ℤ), ((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ) = (k : ℚ) := by  
      have h₇ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).den = 1 := h₅  
      have h₈ : ∃ (k : ℤ), ((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ) = (k : ℚ) := by  
        -- Use the fact that the denominator is 1 to find an integer k  
        -- Use the property of rational numbers with denominator 1  
        have h₉ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).den = 1 := h₅  
        have h₁₀ : ∃ (k : ℤ), ((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ) = (k : ℚ) := by  
          -- Use the fact that the denominator is 1 to find an integer k  
          refine' (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).num, _  
        have h₁₁ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)) = (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).num : ℚ := by  
          have h₁₂ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).den = 1 := h₅  
          have h₁₃ : (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)) = (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ)).num : ℚ :=  
            → by  
              rw [← Rat.num_div_den (((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ))]  
              <|> field_simp [h₁₂]  
              <|> norm_cast  
              <|> simp_all [Rat.den_nz]  
            exact h₁₀  
          exact h₁₁  
          exact h₁₂  
          exact h₁₃  
        obtain (k, h₃) := h₁₃  
      have h₄ : (42 * n : ℤ) ∣ (41 * n + 42 : ℤ) := by  
        have h₅ : ((41 * n + 42 : ℤ) : ℚ) / (42 * n : ℤ) = (k : ℚ) := h₃  
        have h₆ : (42 * n : ℤ) ≠ 0 := by  
          have h₇ : (n : ℕ) > 0 := h₀  
          have h₈ : (42 * n : ℤ) > 0 := by  
            norm_cast  
            <|> nlinarith  
          linearith  
        have h₇ : ((41 * n + 42 : ℤ) : ℚ) = (k : ℚ) * (42 * n : ℤ) := by  
          field_simp [h₆] at h₃  
          <|> ring_nf at h₃ <|> norm_cast at h₃ <|>  
          (try linarith) <|>  
          (try simp_all [mul_comm]) <|>  
          (try ring_nf at h₃ <|> norm_cast at h₃ <|> linarith)  
          <|>  
          (try  
            (try  
              (nlinarith)  
            )  
          )  
          <|>  
          (try  
            (try  
              field_simp at h₃ <|>  
              ring_nf at h₃ <|>  
              norm_cast at h₃ <|>  
              linarith  
            )  
          )  
        have h₈ : (41 * n + 42 : ℤ) = k * (42 * n : ℤ) := by  
          norm_cast at h₇  
          <|>  
          (try ring_nf at h₇ <|> <|>  
            (try norm_num at h₇ <|> <|>  
              (try linarith)  
            )  
          )  
          <|>  
          (try  
            (try  
              field_simp at h₇ <|>
```

Patterns

Proof Style: Linear $\rightarrow$ Tree-like

Generation Strategy: Tactic-Level Generation $\rightarrow$ Incremental Proof & Lemma Generation

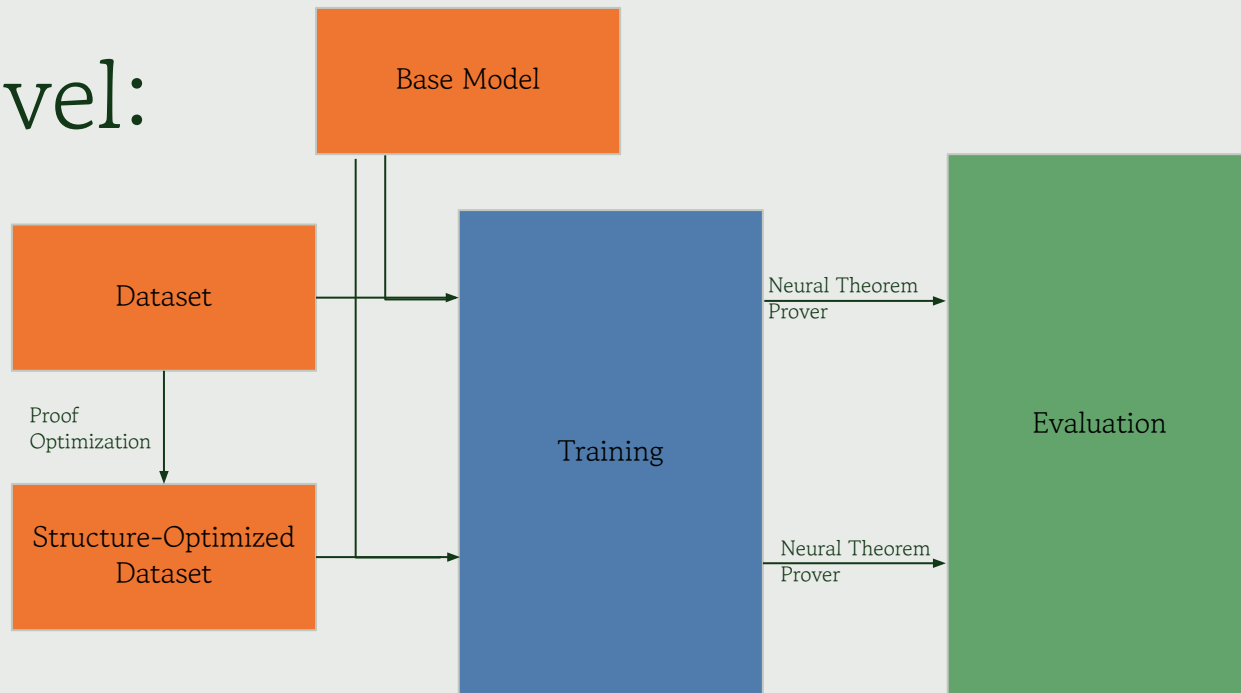
In general, we observe performance increasing and proof “structure” improving correlationally over time. Additionally, the majority of methods begin to converge on a Draft-Sketch-Prove approach w/ lemma generation. So:

- Performance $\sim$ “Proof Shape”?
- Or Performance $\sim$ Generation Strategy?

Is this “Structure”
Useful?

All else being equal, does training on better “structured” proofs result in better downstream theorem prover performance?

High level:



So, how does one structurally optimize a dataset? What does that even mean?

Proof Optimization

Given a theorem statement x in context c with an initial proof y , we want to generate a new proof y' such that $\text{compiles}(c, x, y')$, and given either an *absolute metric* $\Delta : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ or a *relative metric* $\Delta : \mathcal{C} \times \mathcal{X} \times \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ we aim to maximize:

$$\max_{\substack{y \in \mathcal{A}_{\Delta}(c, x) \\ \text{compiles}(c, x, y) = 1 \\ \text{compiles}(c, x, y_0) = 1}} \Delta(c, x; y, y_0)$$

Or, if we want to specify some additional requirements for a “valid proof” for a given metric (i.e. ignore trivial optimizations), we can define an admission set $\mathcal{A}_{\Delta}(c, x)$ and consider the *improvement score*:

$$S_{\Delta}(c, x; y \parallel y_0) = \Delta(c, x; y, y_0) \cdot \text{compiles}(c, x, y) \cdot \text{compiles}(c, x, y_0) \cdot \mathbb{I}\{y \in \mathcal{A}_{\Delta}(c, x)\}$$

Meaning that we would like to optimize for $\max_{y \in \mathcal{Y}} S_{\Delta}(c, x; y \parallel y_0)$,

Note that the task of proof optimization is a proper generalization of neural theorem proving as well as autoformalization.

ImProver

Table 1: Average Proof optimization results.

Metric	Model	Improvement	Nonempty Improvement	Accuracy	Improved Acc.
Length	GPT-4o	3.7	15.15	26.36%	8.31%
	ImProver	20.96	55.29	100.0%	35.44%
Declarativity	GPT-4o	2.21	8.02	18.75%	6.13 %
	ImProver	9.34	30.53	100.0%	24.56%
Mixed	GPT-4o	3.51	23.90	14.70%	5.11%
	ImProver	27.31	39.24	100.0%	30.55

	Improvement	Nonempty Improve.	Accuracy	Improved Acc.
GPT-4o-mini	0	0	3.62%	0%
GPT-4o	7.03	19.67	35.77%	15.33%
+ Output and CoS	8.04 / 6.31	12.38 / 14.17	64.96% / 44.53%	21.17% / 16.06%
+ Example Retrieval	9.34 / 5.67	14.7 / 8.44	63.5% / 67.15%	21.9% / 16.79%
+ Sampling Method	15.35 / 9.34	18.44 / 14.7	83.21% / 63.5%	36.5% / 21.9%
+ n and Model	23.51 / 3.65	26.28 / 4.63	89.47% / 78.95%	45.61% / 8.77%
+ Combos and RAG	34.88 / 28.25	57.56 / 33.48	60.61% / 84.38%	54.55% / 53.12%
ImProver	34.88	57.56	100%	54.55%

Original (human-written)

```
lemma lemma0 {α : Type} {p : α → α → Prop}
  (h1 : ∀ x, ∃! y, p x y)
  (h2 : ∀ x y, p x y ↔ p y x) :
  ∀ x, Classical.choose
    (h1 (Classical.choose (h1
      x).exists)).exists=x := by

-- PROOF START
intro x
obtain ⟨y, h1e, h1u⟩ := h1 x
have h2' : Classical.choose (h1 x).exists =
  y :=
  h1u _ (Classical.choose_spec (h1
    x).exists)
rw [h2']
obtain ⟨w, h1e', h1u'⟩ := h1 y
have h4 := Classical.choose_spec (h1
  y).exists
have h4w : x = w := by
  apply h1u'
  rw [h2]
  exact h1e
rw [h4w]
exact h1u' _ h4
```

ImProver (length-optimized)

```
lemma lemma0 {α : Type} {p : α → α → Prop}
  (h1 : ∀ x, ∃! y, p x y)
  (h2 : ∀ x y, p x y ↔ p y x) :
  ∀ x, Classical.choose
    (h1 (Classical.choose (h1
      x).exists)).exists=x := by

-- PROOF START
intro x
obtain ⟨y, h1e, h1u⟩ := h1 x
rw [h1u _ (Classical.choose_spec _)]
obtain ⟨w, h1e', h1u'⟩ := h1 y
rw [h1u' _ ((h2 _ _).mpr h1e)]
exact h1u' _ (Classical.choose_spec _)
```

ProofOptimizer

Evaluation Metrics Given an original proof p and k candidate simplifications generated by the model, $p'_1, p'_2, \dots, p'_k$, we define $l_i = \min(|p|, |p'_i|)$ if p'_i is a valid proof and $l_i = |p|$ otherwise. (Intuitively, an invalid attempt reverts to the original proof length). We evaluate proof simplification using two metrics:

- $\min@k \triangleq \min_i \{l_i\}$ denotes the minimum shortened proof length (lower is better).
- $\text{red}@k \triangleq \max_i \left\{ \frac{|p| - l_i}{|p|} \right\} = 1 - \frac{\min@k}{|p|}$ denotes the maximum relative proof length reduction from the original proof (higher is better).

Table 1: **Min@k and Red@k throughout expert iteration and online RL.** Our RL model has strong @1 results, while our Explt model has strong @32 results. RL metrics are Gaussian-smoothed.

Dataset	Category	Model	Min@1 ↓	Min@32 ↓	Red@1 ↑	Red@32 ↑
miniF2F		<i>Linted</i>		302		0.0%
		Gemini-2.5-Pro	280	207	24.3%	57.2%
		Gemini-2.5-Pro + States	283	207	26.4%	58.7%
		Base (7B)	283	202	17.6%	56.2%
	<i>Explt</i>	Base + It 1	266	178	33.4%	67.0%
		Base + It 2	251	166	45.1%	70.6%
		ProofOptimizer-Explt	241	153	49.0%	72.3%
	<i>RL</i>	ProofOptimizer-RL	190	152	63.6%	70.9%
		It 2 + Test-Time RL	160	154	72.5%	73.4%
Putnam Bench		<i>Linted</i>		1359		0.0%
		Gemini-2.5-Pro	1348	1303	5.5%	18.0%
		Gemini-2.5-Pro + States	1371	1319	6.1%	19.2%
		Base (7B)	1341	1222	3.9%	20.5%
	<i>Explt</i>	Base + It 1	1341	1215	5.2%	22.5%
		Base + It 2	1335	1186	6.9%	24.7%
		ProofOptimizer-Explt	1328	1161	8.2%	26.3%
	<i>RL</i>	ProofOptimizer-RL	1303	1258	14.9%	21.1%
		It 2 + Test-Time RL	1260	1255	23.8%	24.2%

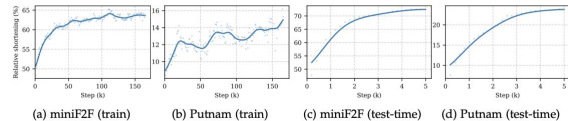


Figure 2: **Evolution of proof reduction (red@1) during RL training (a, b) and test-time RL (c, d).** We use Gaussian smoothing ($\sigma = 5$ evaluation intervals for RL training and $\sigma = 3$ for test-time RL). See Fig. 9 for the corresponding red@32 metrics.

ImProver²

Objectives:

1. Build a general method that allows for simple proof optimization to arbitrary metrics
2. Test this method on real-world, utility-focused metrics
3. Build a training pipeline that can optimize many different metrics without much/any manual tuning on small parameter models, that allows them to perform competitively with much larger models.
4. Do all of the above on research math

Original (human-written)

```
theorem isCoatom_iff [OrderTop A] {K : A} :  
  IsCoatom K  $\leftrightarrow$  K  $\neq$   $\wedge$   $\forall$  H g, K  $\leq$  H  $\rightarrow$  g  $\notin$   
    K  $\rightarrow$  g  $\in$  H  $\rightarrow$  H = := by  
simp_rw [IsCoatom, lt_iff_le_not_le,  
  SetLike.not_le_iff_exists,  
  and_comm (a := _  $\leq$  _), and_imp,  
  exists_imp,  $\leftarrow$  and_imp, and_comm]
```

ImProver² (dependency-optimized)

```
theorem isCoatom_iff [OrderTop A] {K : A} :  
  IsCoatom K  $\leftrightarrow$  K  $\neq$   $\wedge$   $\forall$  H g, K  $\leq$  H  $\rightarrow$  g  $\notin$   
    K  $\rightarrow$  g  $\in$  H  $\rightarrow$  H = := by  
change (K  $\neq$   $\wedge$   $\forall$  H, K  $<$  H  $\rightarrow$  H = )  $\leftrightarrow$  K  $\neq$   
   $\wedge$   $\forall$  H g, K  $\leq$  H  $\rightarrow$  g  $\notin$  K  $\rightarrow$  g  $\in$  H  $\rightarrow$  H =  
constructor  
· intro ⟨hne, h⟩  
  constructor  
  · exact hne  
  · intro H g hKH hgnotK hgH  
    have : K < H := ⟨hKH, fun HleK =>  
      hgnotK (HleK hgH)⟩  
    exact h H this  
· intro ⟨hne, h'⟩  
  constructor  
  · exact hne  
  · intro H hlt  
    have hle : K  $\leq$  H := hlt.1  
    have hnot :  $\neg$  H  $\leq$  K := hlt.2  
    have ⟨g, hgH, hgnotK⟩ :  $\exists$  x, x  $\in$  H  $\wedge$  x  $\notin$   
      K := (SetLike.not_le_iff_exists :  $\neg$  H  $\leq$   
        K  $\leftrightarrow$   $\exists$  x  $\in$  H, x  $\notin$  K).mp hnot  
    exact h' H g hle hgnotK hgH
```

Figure 17: Optimizing SetLike.isCoatom_iff from Mathlib.Order.Atoms for dependencies

ImProver²

Objectives:

1. Build a general method that allows for simple proof optimization to arbitrary metrics
2. Test this method on real-world, utility-focused metrics
3. Build a training pipeline that can optimize many different metrics without much/any manual tuning on small parameter models, that allows them to perform competitively with much larger models.
4. Do all of the above on research math

Original (human-written)

```
lemma disjoint_of_singleton_inter_left_wo {X
  Y : Set  $\alpha$ } {a :  $\alpha$ } (hXY :  $X \cap Y = \{a\}$ ) :
   $X \setminus \{a\} \cap Y :=$  by
  tauto_set
```

ImProver² (modularity-optimized)

```
lemma disjoint_of_singleton_inter_left_wo {X
  Y : Set  $\alpha$ } {a :  $\alpha$ } (hXY :  $X \cap Y = \{a\}$ ) :
   $X \setminus \{a\} \cap Y :=$  by
  -- from the equality of sets, extract the
  -- characterization of membership in the
  -- intersection
  have h_mem :  $\forall x, x \in X \cap Y \leftrightarrow x = a :=$  by
    intro x
    rw [hXY]
    simp

  -- show the intersection of  $X \setminus \{a\}$  with  $Y$ 
  -- is empty
  have h_empty :  $(X \setminus \{a\}) \cap Y = \emptyset :=$  by
    ext x
    constructor
    · rintro ⟨⟨hxX, hne⟩, hy⟩
      -- x belongs to  $X \cap Y$ , hence equals a,
      -- contradicting  $x \neq a$ 
      have :  $x \in X \cap Y :=$  ⟨hxX, hy⟩
      have := (h_mem x).1 this
      exact hne this
    · intro h; cases h

  -- disjointness is equivalent to
  -- intersection being empty
  exact (Set.disjoint_iff_inter_eq_empty.mpr
    h_empty)
```

Figure 17: Optimizing disjoint_of_singleton_inter_left_wo from Seymour.ForMathlib.SetTheory for modularity

Proof Modularity

Proof Modularity is the number of *spawned goals*, that are:

- critical to the correctness of the proof
- nonduplicates
- nontrivial

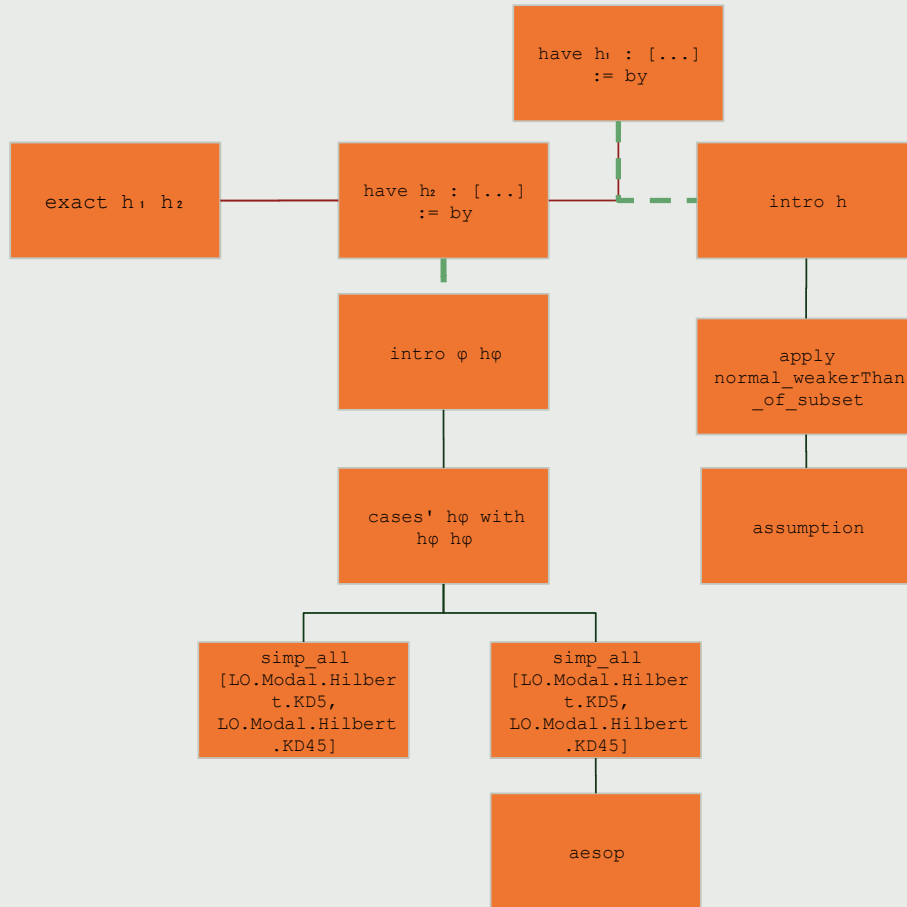
```
lemma KD5_weakerThan_KD45 : (Hilbert.KD5  $\alpha$ )  $\leq$  (Hilbert.KD45  $\alpha$ ) := by
  -- Introduce a lemma to handle the subset relationship between the axioms of KD5 and KD45
  have h1 : (LO.Modal.Hilbert.KD5  $\alpha$ ).axioms  $\subseteq$  (LO.Modal.Hilbert.KD45  $\alpha$ ).axioms  $\rightarrow$ 
    (Hilbert.KD5  $\alpha$ )  $\leq$  (Hilbert.KD45  $\alpha$ ) := by
    intro h
    -- Apply the lemma that establishes the weakening relation given a subset of axioms
    apply normal_weakerThan_of_subset
    -- Use the given subset relation to conclude the proof
    <=> assumption
  -- Prove the subset relation between the axioms of KD5 and KD45
  have h2 : (LO.Modal.Hilbert.KD5  $\alpha$ ).axioms  $\subseteq$  (LO.Modal.Hilbert.KD45  $\alpha$ ).axioms := by
    -- Introduce the axioms of KD5 and verify they are included in the axioms of KD45
    intro  $\varphi$  h $\varphi$ 
    cases' h $\varphi$  with h $\varphi$  h $\varphi$ 
    <=> simp_all [LO.Modal.Hilbert.KD5, LO.Modal.Hilbert.KD45]
    <=> aesop
  -- Combine the results to conclude the weakening relation
  exact h1 h2
```

Proof Modularity

Proof Modularity is the number of *spawned goals*, that are:

- critical to the correctness of the proof
- nonduplicates
- nontrivial

```
lemma KD5_weakerThan_KD45 : (Hilbert.KD5 α) ≤ (Hilbert.KD45 α) := by
  -- Introduce a lemma to handle the subset relationship between the axioms of KD5 and KD45
  have h₁ : (LO.Modal.Hilbert.KD5 α).axioms ⊆ (LO.Modal.Hilbert.KD45 α).axioms →
    (Hilbert.KD5 α) ≤ (Hilbert.KD45 α) := by
    intro h
    -- Apply the lemma that establishes the weakening relation given a subset of axioms
    apply normal_weakerThan_of_subset
    -- Use the given subset relation to conclude the proof
    <|> assumption
    -- Prove the subset relation between the axioms of KD5 and KD45
    have h₂ : (LO.Modal.Hilbert.KD5 α).axioms ⊆ (LO.Modal.Hilbert.KD45 α).axioms := by
      -- Introduce the axioms of KD5 and verify they are included in the axioms of KD45
      intro φ hφ
      cases' hφ with hφ hφ
      <|> simp_all [LO.Modal.Hilbert.KD5, LO.Modal.Hilbert.KD45]
      <|> aesop
    -- Combine the results to conclude the weakening relation
    exact h₁ h₂
```

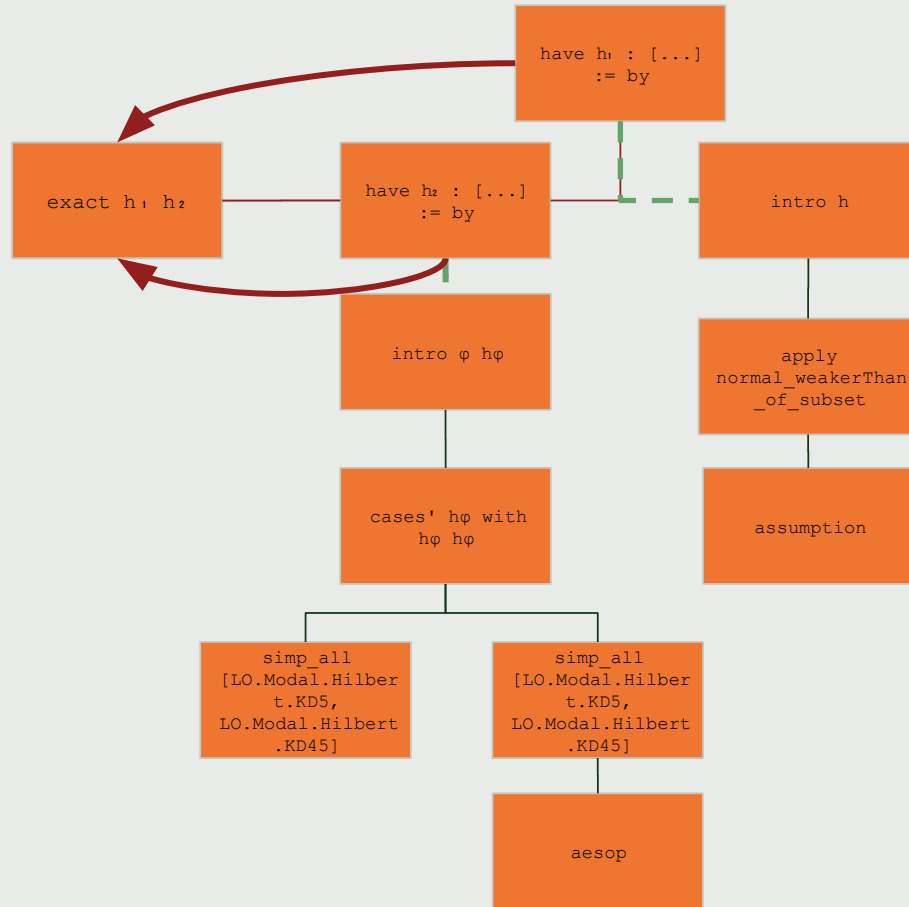


Proof Modularity

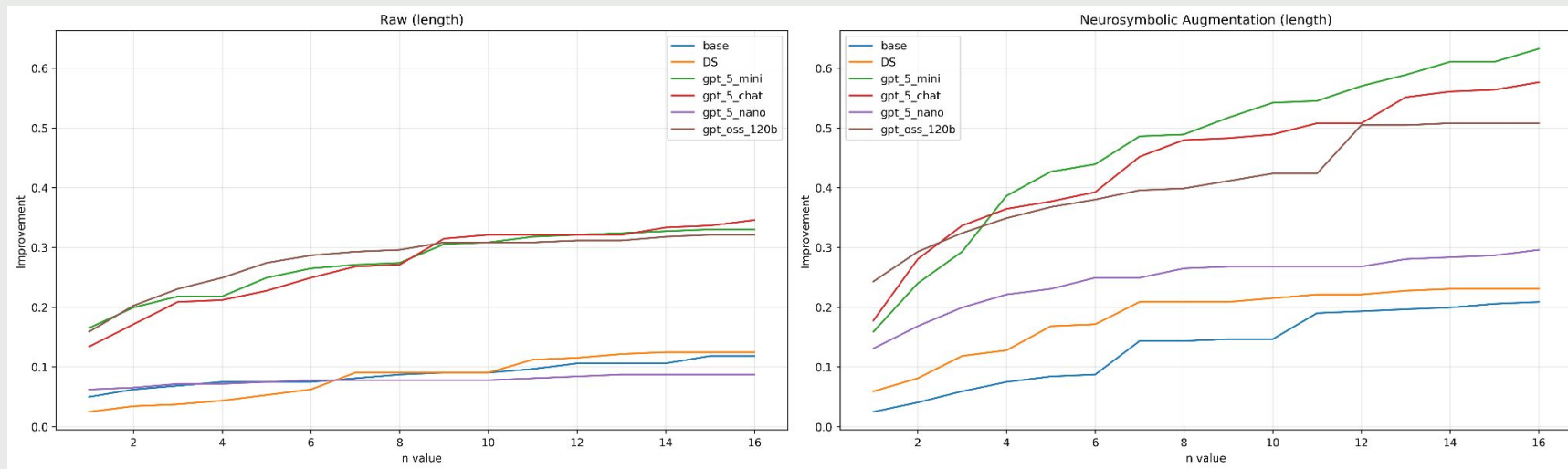
Proof Modularity is the number of *spawned goals*, that are:

- critical to the correctness of the proof
- nonduplicates
- nontrivial

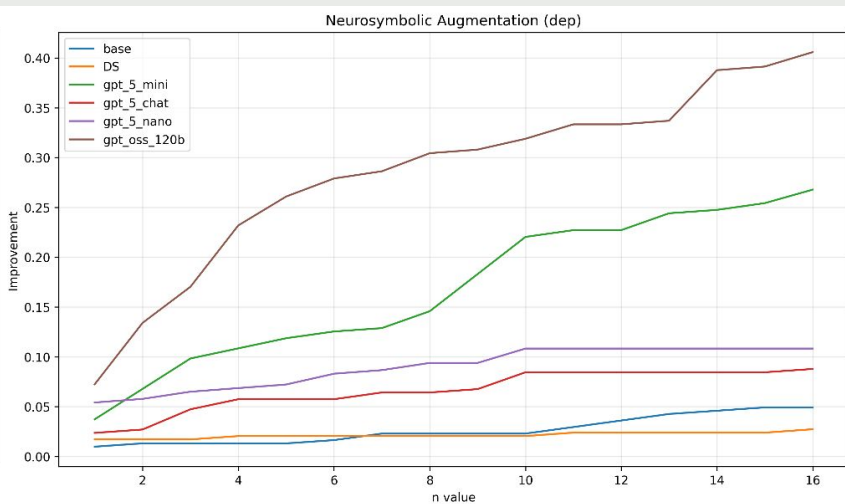
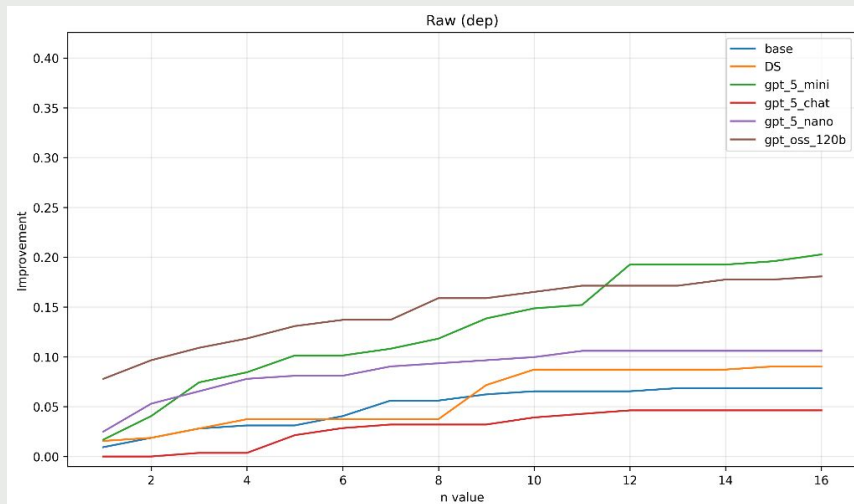
```
lemma KD5_weakerThan_KD45 : (Hilbert.KD5 α) ≤ (Hilbert.KD45 α) := by
  -- Introduce a lemma to handle the subset relationship between the axioms of KD5 and KD45
  have h₁ : (LO.Modal.Hilbert.KD5 α).axioms ⊆ (LO.Modal.Hilbert.KD45 α).axioms →
    (Hilbert.KD5 α) ≤ (Hilbert.KD45 α) := by
    intro h
    -- Apply the lemma that establishes the weakening relation given a subset of axioms
    apply normal_weakerThan_of_subset
    -- Use the given subset relation to conclude the proof
    <|> assumption
    -- Prove the subset relation between the axioms of KD5 and KD45
    have h₂ : (LO.Modal.Hilbert.KD5 α).axioms ⊆ (LO.Modal.Hilbert.KD45 α).axioms := by
      -- Introduce the axioms of KD5 and verify they are included in the axioms of KD45
      intro φ hφ
      cases' hφ with hφ hφ
      <|> simp_all [LO.Modal.Hilbert.KD5, LO.Modal.Hilbert.KD45]
      <|> aesop
    -- Combine the results to conclude the weakening relation
    exact h₁ h₂
```



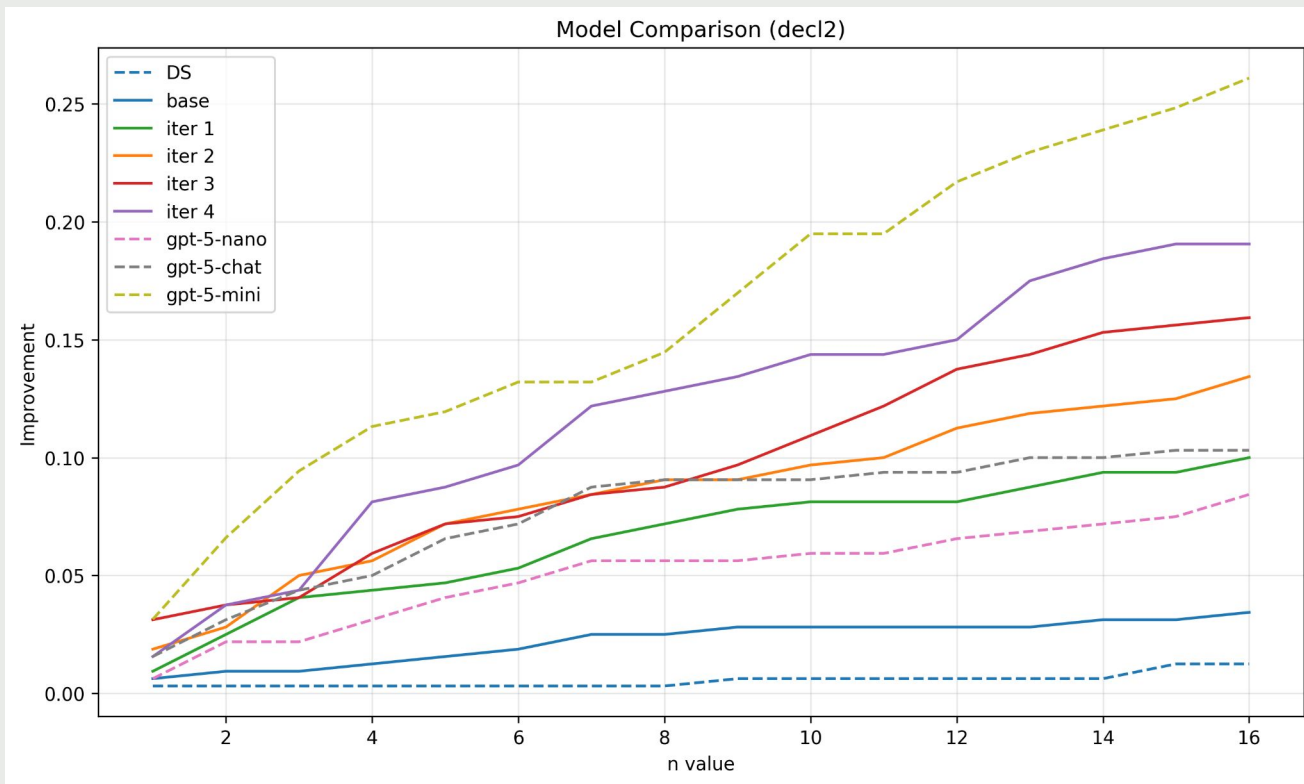
(Preliminary) Results



(Preliminary) Results

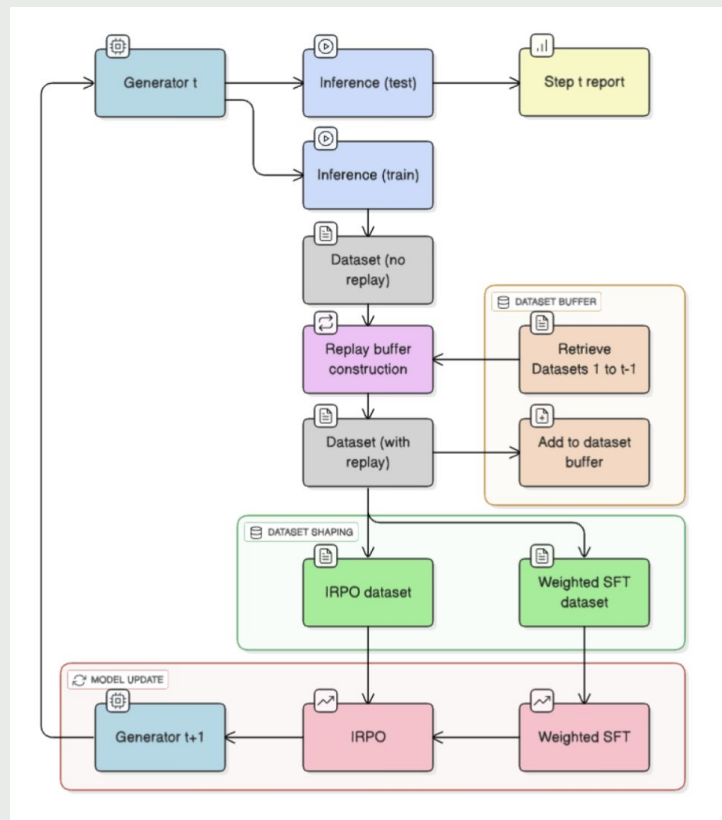


(Preliminary) Results



Training Pipeline

- Initialize with SFT
- IRPO: Iterative Reasoning Policy Optimization
 - With dynamic replay and difficulty filtering
 - Off policy, and more stable than DPO, GRPO, etc.



Effect on downstream theorem proving

